THREADS REFERENCE SHEET.

Threads share memory → shared memory creates race conditions => sync mechanisms solve them.

THREADS – concurrent execution
MUTEXES – PREVENT race conditions
COND. VAR. – wait efficiently for events (allow one thread)
RW-LOCKS – optimise many-readers scenarios

SEMAPHORES – allow more threads
BARRIERS – everybody waits for everybody

Create thread:

```
pthread_create(&tid, NULL, function, arg)
```

Thread function:

```
void *f(void *arg)
```

join thread (wait)

```
pthread_join(tid, NULL)
```

static mutex:

```
pthread_mutex_t mymutex = PTHREAD_MUTEX_INITIALIZER
```

dynamic mutex:

```
pthread_mutex_init();
```

many reads => rwlock
shared var? => mutex
turns/alternating? => cond.var
all wait -> barrier
max n inside => semaphore

Conditional variables.

THREAD that must wait: ↓

```
pthread_mutex_lock(&m);
while (condition_is_false){
    pthread_cond_wait(&cond, &m);
}
pthread_mutex_unlock(&m);
```

THREAD that changes the condition: ↓

```
pthread_mutex_lock(&m);
change_condition; ↴
pthread_cond_signal(&cond);
pthread_mutex_unlock(&m);
```

pthread_cond_wait(&cond, &m)
↳ • unlocks mutex
  • sleeps
  • wakes later
  • locks mutex again.

wait = „sleep until another thread tells me the condition changed".

CONDITIONAL VAR • „thread waits for its turn"
• used for: alternating, ordering; works with mutexes
• Functions:

pthread_cond_wait(&cond, &mutex)
pthread_cond_signal(&cond) ← wake one thread
pthread_cond_broadcast(&cond) ← wake all threads that are waiting

SEMAPHORE: • „allow n threads simultaneously"
• Functions:

sem_init(&sem, 0, N)	→ n threads at once
sem_wait(&sem)	→ car tries to enter
sem_post(&sem)	→ car leaves

BARRIER: • „everybody waits for everybody"
• Functions:

pthread_barrier_init(&barrier, NULL, n) ← n runners must arrive
pthread_barrier_wait(&barrier)

↳ wait until n'th one arrives.

MUTEX
• „one thread at a time" (one key)
• used for: shared var, arrays, counters
• Functions:

pthread_mutex_lock(&mutex)
pthread_mutex_unlock(&mutex)

• Init: pthread_mutex_t

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER
```

```
pthread_mutex_lock(&m)
  //critical section
pthread_mutex_unlock(&m)
```

RWLOCK: • „many reads, one write"

```
pthread_rwlock_rdlock(&rwlock)
pthread_rwlock_wrlock(&rwlock)
pthread_rwlock_unlock(&rwlock)
```

JOIN: • wait for thread to finish: `pthread_join(thread, NULL)`

preemption